

Modularização de Código e Avaliação de LLMs

Aluna: Julia S. Rios Curso: Eng. Mecânica

Pseudocódigo:

- 1- Ler os valores de temperaturas;
- 2- Excluir valores que estiverem abaixo de -20°C ou acima de 200°C;
- 3- Descobrir qual o menor valor até o maior, temperatura média e variação;
- 4- Classificar as temperaturas: alerta >90 e crítico >120;
- 5- Contar quantas temperaturas ficaram em estado crítico;
- 6- Encontrar a maior sequência de temperaturas críticas;
- 7- Gerar um relatório de todas as informações calculadas.

(Os códigos gerados pelas LLMs serão anexados separadamente desse documento)

LLM A= ChatGPT

LLM B= Gemini

#	Critério de modularização	LLM A	LLM B
1	Decomposição em funções, cada uma com responsabilidade única	1 Funções bem separadas, mas gerar_relatorio acumula cálculo e exibição ao invocar outras funções internamente.	2 Cada função tem papel único e claro. fluxo_principal só orquestra, sem executar lógica própria.
2	Nomes descritivos de funções e variáveis	2 Nomes de funções claros. Variáveis como atual e maior em maior_sequencia_critica poderiam ser mais específicas, mas não prejudicam.	2 Nomes consistentes e expressivos em funções e variáveis (sequencia_atual, dados_filtrados, total_criticos).
3	Ausência de duplicação de código (DRY)	0 A condição temp > 120 aparece em três funções distintas sem reaproveitamento de classificar_temperatura.	2 analisar_criticos reutiliza classificar_temperatura para checar criticidade — sem repetição de lógica.
4	Parâmetros e retornos bem definidos; evita variáveis globais	1 Retornos bem definidos, mas o fluxo principal usa a variável temperaturas no escopo global, fora de qualquer função.	2 Zero variáveis globais. Todo o estado flui por parâmetros e retornos explícitos dentro de fluxo_principal.
5	Reaproveitamento de funções (ex.:	1 Funções são chamadas entre si, mas a validação de intervalo	2 filtrar_temperaturas isola a validação.

	validação usada em um só lugar)	(-20 a 200) está embutida em ler_temperaturas sem função separada reutilizável.	classificar_temperatura é reaproveitada em duas funções diferentes.
6	Existência de uma função principal organizando o fluxo	0 Fluxo no escopo global, sem encapsulamento. Não há main nem guarda if __name__ == "__main__".	2 fluxo_principal estrutura o pipeline completo, protegida por if __name__ == "__main__".
7	Legibilidade geral e comentários úteis (sem excesso)	1 Código legível e bem estruturado visualmente, mas sem docstrings ou comentários de qualquer tipo.	2 Docstrings em todas as funções, objetivas. Comentários inline explicam decisões não óbvias sem poluir o código.
	Total (máx. 7)	3	7